

NLP Lab 1 — Text Preprocessing

Objective: Practice text preprocessing techniques in Python for Natural Language Processing tasks.

1) Sample texts to practice on

```
texts = [
    "I LOVED this movie!!! ■■ Check review: https://example.com/rev?id=123",
    "Email me at john.doe@example.org and follow @john_doe #productivity",
    "The price was 1,299.99 USD—what a deal!!",
    "We're testing contractions: don't, can't, I'm; plus accents: naïve, café.",
    "Boring... soooo boring... 0000 times."
]
```

2) Lowercasing + Unicode normalization + basic cleanup

```
import re
import unicodedata

def normalize_text(s: str) -> str:
    s = unicodedata.normalize("NFKC", s)
    s = s.lower()
    s = s.replace("'", "").replace('"', '').replace(" ", '').replace("-", "-").replace("—", "-")
    s = re.sub(r"\s+", " ", s).strip()
    return s

clean1 = [normalize_text(t) for t in texts]
```

3) De-noising: remove URLs, emails, @mentions, hashtags

```
def denoise(s: str) -> str:
    s = re.sub(r"https?://\S+|www\.\S+", " ", s)
    s = re.sub(r"\b[\w\.-]+@[ \w\.-]+\.\w+\b", " ", s)
    s = re.sub(r"@ \w+", " ", s)
    s = re.sub(r"#(\w+)", r"\1", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

clean2 = [denoise(t) for t in clean1]
```

4) Punctuation & numbers handling

```
def strip_punct(s: str) -> str:
    return re.sub(r"^[^\w\s']+", " ", s)

def remove_numbers(s: str) -> str:
    return re.sub(r"\d+", " ", s)

clean3 = [remove_numbers(strip_punct(t)) for t in clean2]
```

5) Tokenization

```
from nltk.tokenize import word_tokenize
tokenized = [word_tokenize(t) for t in clean3]
```

6) Stop-word removal

```
from nltk.corpus import stopwords
stop = set(stopwords.words("english"))
def remove_stopwords(tokens):
    return [w for w in tokens if w not in stop]

no_stop = [remove_stopwords(toks) for toks in tokenized]
```

7) Stemming vs Lemmatization

```

from nltk.stem import PorterStemmer, WordNetLemmatizer

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def stem_tokens(tokens):
    return [stemmer.stem(w) for w in tokens]

def lemmatize_tokens(tokens):
    return [lemmatizer.lemmatize(w) for w in tokens]

stemmed = [stem_tokens(toks) for toks in no_stop]
lemmatized = [lemmatize_tokens(toks) for toks in no_stop]

```

8) Handle emojis/emoticons (optional)

```

EMO_MAP = { "❤️": "love", "🔥": "fire", "😂": "funny", "😭": "sad", "❤️🔥": "love" }

def replace_emojis(s: str) -> str:
    for k,v in EMO_MAP.items():
        s = s.replace(k, f" {v} ")
    s = re.sub(r"^[^w\s']+", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

emoji_demo = [replace_emojis(t) for t in texts]

```

9) End-to-end preprocessing pipeline

```

def preprocess_pipeline(
    s: str,
    keep_numbers: bool = True,
    remove_stop: bool = True,
    do_stem: bool = False,
    do_lemma: bool = True,
    handle_emoji: bool = False
):
    if handle_emoji:
        s = replace_emojis(s)
    s = normalize_text(s)
    s = denoise(s)
    s = strip_punct(s)
    if not keep_numbers:
        s = remove_numbers(s)
    toks = word_tokenize(s)
    if remove_stop:
        toks = [w for w in toks if w not in stop]
    if do_stem:
        toks = stem_tokens(toks)
    if do_lemma:
        toks = lemmatize_tokens(toks)
    return toks

processed = [preprocess_pipeline(t, keep_numbers=False, handle_emoji=True) for t in texts]

```

Practice Tasks: 1. Modify the pipeline to keep numbers but strip punctuation, and test on the sample texts. 2. Add a rule to preserve currency patterns like "\$1,299.99" while removing other numbers. 3. Expand EMO_MAP with at least 10 emoji-to-word mappings and measure how many tokens change. 4. Add a contraction expander (e.g., "don't" -> "do not"). 5. Create two versions of the corpus: stemmed vs lemmatized; compare vocabulary sizes.